

POD-Attention: 解锁完整的预填充解码重叠以实现更快的 LLM 推理

Aditya K Kamath*
华盛顿大学 美国西雅图

Ramya Prabhu
微软研究院印度班加罗尔

Jayashree Mohan
微软研究院印度班加罗尔

西蒙彼得华盛顿大学美国西雅图

Ramachandran Ramjee
微软研究院印度班加罗尔

Ashish Panwar
微软研究院印度班加罗尔

抽象的

LLM 推理中的每个请求都会经历两个阶段：计算限制的预填充和内存带宽限制的解码。为了提高 GPU 利用率，最近的系统使用混合批处理，将不同请求的预填充和解码阶段合并到同一批处理中。这种方法优化了线性运算，但注意力计算仍然效率低下，因为现有的注意力内核专门针对预填充和解码阶段独立执行。

在本文中，我们提出了 POD-Attention——第一个能够有效计算混合批次注意力的 GPU 内核。POD-Attention 旨在通过仔细分配 GPU 资源，使预填充和解码操作在同一多处理器上同时发生，从而最大限度地利用计算和内存带宽。与使用独立优化的预填充和解码注意力内核相比，POD-Attention 可将注意力计算速度提高高达 59%（平均 28%），从而实现更高的吞吐量和更低的延迟 LLM 推理。

CCS 概念：• 计算方法 → 机器学习；
• 计算机系统组织 → 并行体系结构。

关键词：大语言模型；GPU；自我关注

ACM参考格式：

Aditya K Kamath, Ramya Prabhu, Jayashree Mohan, Simon Peter, Ramachandran Ramjee 和 Ashish Panwar. 2025. POD-Attention: 解锁完整的预填充解码重叠以实现更快的 LLM 推理. 第 30 届 ACM 编程语言和操作系统架构支持国际会议论文集, 第 2 卷 (ASPLOS '25), 2025 年 3 月 30 日至 4 月 3 日, 荷兰鹿特丹. ACM, 美国纽约州纽约市, 16 页. <https://doi.org/10.1145/3676641.3715996>

*Work done as an intern at Microsoft Research India.



This work is licensed under a Creative Commons Attribution 4.0 International License.

ASPLOS '25, March 30-April 3, 2025, Rotterdam, Netherlands

© 2025 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-1079-7/2025/03

<https://doi.org/10.1145/3676641.3715996>

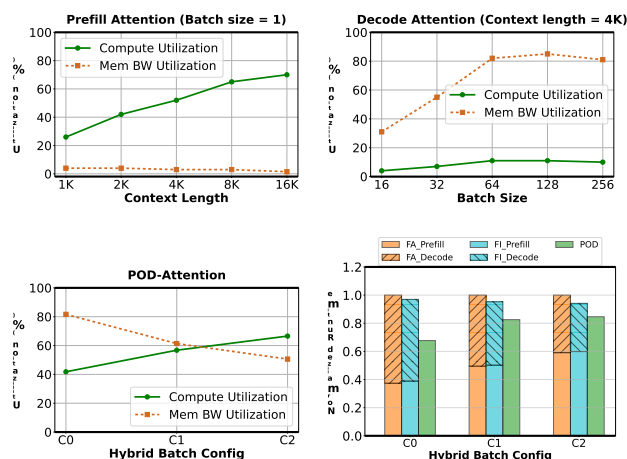


图 1. 最先进的注意力内核利用计算或内存（FA: Flash Attention, FI: FlashInfer）。POD-Attention 利用计算和内存来加速混合批次中的注意力计算（配置见表 1。型号：2 个 A100 GPU 上的 Llama-3-8B）。

1 简介

为大型语言模型 (LLM) 提供服务的基础设施正在不断扩展，以满足其不断增长的需求 [3, 16]。大型服务提供商通常依赖昂贵的高端 GPU 来满足峰值需求或延迟目标 [46]。因此，优化 LLM 服务系统变得至关重要 [21, 23, 41, 57, 62, 65, 66]。部署的整体效率取决于 GPU 资源的利用情况。

从资源利用的角度来看，LLM 推理是一项具有挑战性的工作负载，因为不同阶段在不同时间需要不同的资源 [22–24, 66]。LLM 请求的处理从高度并行（因此受计算限制）预填充阶段开始，然后是受内存限制的解码阶段 [24]。因此，高效地为大语言模型服务需要高计算能力和高内存带宽。理想的系统将努力最大化计算和内存的利用率。然而，这样做并不简单，因为对于给定的请求，预填充和解码阶段发生在不同的时间。

最先进的 LLM 服务系统通过将不同请求的预填充和解码阶段的输入组合到同一批处理中来应对这一挑战[24,33,62]——我们将这种技术称为混合批处理。混合批处理无需单独从 GPU 高带宽内存 (HBM) 获取模型权重以进行预填充和解码令牌。相反，它允许 GPU 一次获取模型权重，并使用它们来计算预填充和解码输入。混合批处理还有助于减少尾延迟：为了限制每次迭代的运行时间，调度程序可以将长输入提示（预填充输入）划分为多个较小的块，然后在每次迭代时将正在进行的解码与新的预填充块结合起来 [23, 33]。因此，混合批处理的使用在当今各种 LLM 服务系统中很常见 [23,33,41,62, 66]。

虽然之前的工作主要集中在优化线性运算 [23,33,62]，但它们并没有优化混合批次的注意力计算。这对于主要处理小上下文长度的系统来说是合理的，因为在此设置中线性操作主导运行时间[62, 66]。相反，随着上下文长度的增加，注意力计算成为主要的性能瓶颈（图4）。

最近的一些工作也尝试优化注意力计算[30,31,34,48]，但当前的解决方案分别解决预填充和解码操作——最大化预填充的计算利用率和解码的带宽利用率，如图1所示。在本文中，我们表明这种方法不是最理想的，因为它使得关键GPU资源在计算的不同部分没有得到充分利用。例如，图 1 表明预填充注意力内核的内存带宽利用率通常低于 5%，而解码注意力内核的计算利用率低于 10%。使用独立优化的内核的效果对于混合批处理尤其明显，因为预填充和解码内核立即相继执行，导致资源的高需求时期紧随其后的是同一资源的低利用率。

为了提高混合批处理的效率，我们提出了 POD-Attention——据我们所知，第一个 GPU 内核，它可以有效地批处理预填充和解码注意力的计算。在此过程中，我们首先证明 (§3)，由于各种限制，例如落后线程、同步障碍以及缺乏 GPU 流多处理器 (SM) 上不同协作线程阵列 (CTA) 的 SM 级共置保证，现有技术在融合注意力计算方面无法提供足够的性能。POD-Attention 通过以 CTA 并行方式融合计算、在 GPU 内引入基于 SM 感知的基于软件的 CTA 调度来解决这些问题 (§4)。POD-Attention 以最先进的 FlashAttention 内核 [1] 为基础，根据给定批次请求的要求利用计算和内存资源，显着加速注意力计算（见图 1）。

Config.	Prefill			Decode		Resource requirement
	BS	CS	CL	BS	CL	
C0	1	1K	12K	80	12K	memory-bound
C1	1	12K	12K	220	12K	balanced
C2	1	16K	16K	250	12K	compute-bound

表 1. 图 1 中评估的混合批次的详细信息（BS：批次大小，CS：块大小，CL：上下文长度）。

我们还将 POD-Attention 集成到最先进的 LLM 推理调度程序 Sarathi-Serve [23] 中。我们的实验表明，POD-Attention 计算注意力的速度比 FlashAttention 和 FlashInfer 的预填充和解码注意力内核快 59%（平均 28%）。在端到端 LLM 推理性能方面，POD-Attention 将吞吐量提高了高达 22%，同时还减少了关键延迟指标，例如首次令牌时间 (TTFT)、令牌之间时间 (TBT) 以及 Sarathi-Serve 的端到端请求执行延迟。

贡献：我们做出以下贡献：

- 我们强调，独立优化预填充和解码注意内核对于基于混合批处理的 LLM 推理来说并不是最佳选择。
- 我们提出了 POD-Attention——一个 GPU 内核，它同时计算预填充和解码注意力，以同时利用计算和内存带宽。
- 我们将 POD-Attention 集成到 Sarathi-Serve 中，并表明与使用独立优化的预填充和解码注意内核相比，它可以实现高吞吐量和低延迟的 LLM 推理。

2 背景和动机

我们首先讨论为什么 LLM 服务系统使用混合批处理，然后激发优化注意力计算的需求。最后，我们概述了 GPU 执行。

2.1 大语言模型 (LLM) 推理

LLM 将用户输入和输出作为标记进行处理，在内部表示为向量。推理过程中的每个请求都会经历两个阶段——预填充和解码[62]。预填充阶段并行处理用户提示的令牌并生成第一个输出令牌，其延迟称为首次令牌时间 (TTFT)。随后，解码阶段在每次迭代自回归中生成一个输出令牌（每个请求）。生成每个输出令牌所需的延迟称为令牌之间的时间 (TBT)。预填充阶段是高度并行和计算限制的，而解码阶段是内存限制的。由于大量令牌的并行处理，预填充迭代的延迟通常高于解码迭代的延迟。

预填充和解码操作的独特计算特性在 LLM 推理中产生吞吐量与延迟的权衡 [23,35,46,65]，如图 2 所示。

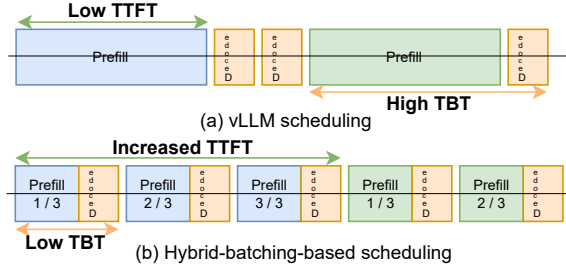


图 2. 调度策略对 TTFT 和 TBT 的影响。

解码受内存限制，使用大批量可以提高吞吐量。原始的 vLLM 调度程序 [41] 使用预填充优先调度来最大化解码批量大小（图 2(a)）。这种方法提供了低 TTFT，但代价是高 TBT，因为新请求的预填充可能会暂停正在进行的解码，导致生成停滞 [23]。高 TBT 在长上下文场景中尤其成问题，其中每次生成停顿可能持续几秒钟。

高 TBT 问题已在现实世界的部署中得到承认 [14]。Sarathi-Serve [23] 提出了与连续混合批处理相结合的分块预填充 [62]——一种将请求的预填充令牌划分为多个较小块的技术，并通过持续解码在每次迭代中安排一个预填充块（图 2(b)）。通过这种方式，Sarathi-Serve 可以增加批量大小，同时避免生成停顿，从而提高性能和用户交互性。各种 LLM 服务系统都采用了这种技术 [2, 64, 66]，包括 vLLM [18]。

在混合批处理的常见情况下，执行的批处理由一个预定大小的预填充块和多个解码组成（如表 1 所示）。例如，考虑一个工作负载，其中每个请求包含 2K 预填充令牌并生成 200 个输出（解码）令牌。如果预填充块大小为 1K，则请求的预填充将通过两次迭代完成（预填充令牌/块大小）。预填充阶段完成后，它必须执行另外 200 次迭代——每次迭代对应一个输出令牌。在这 200 次迭代中，100 个请求可以完成其预填充阶段以加入正在运行的批次。这导致稳定状态下的有效批量大小为 101，其中 100 个请求在其解码阶段与新请求的一个预填充块一起执行。执行这些混合批次需要高计算量（对于预填充块）和高内存带宽（对于解码请求）。

图 3 显示了混合批处理在实践中的工作原理。除了注意力之外，所有其他操作都是线性的，即按元素计算。线性运算遵循规则 $f(x + y) = f(x) + f(y)$ ，因此可以组合线性运算的输入，通过相同的模型权重进行计算以减少内存访问，然后进行分离。相反，注意力是一个序列级运算符，在三个之间计算

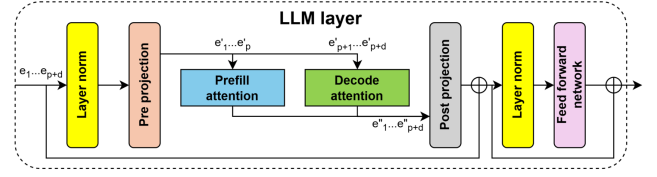


图 3. 混合批次计算。当前系统一起计算预填充输入 ($e_1 \dots e_p$) 和解码输入 ($e_{p+1} \dots e_{p+d}$) 以进行线性运算。然而，他们使用专门的内核分别计算预填充和解码注意力。

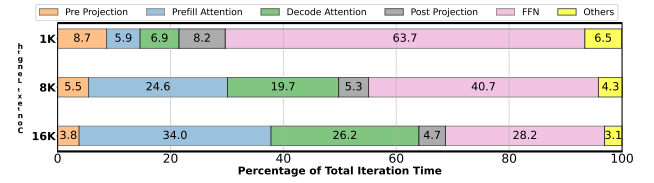


图 4. 混合批处理迭代运行时中不同操作的贡献（模型：Llama-3-8B，批大小：60，块大小：1K）。对于每个上下文长度，我们显示处理提示的最后一个块的迭代运行时间。

表示 Q（查询，当前正在处理的标记）和 K/V（键/值，到目前为止所看到的序列中的所有标记）：

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\text{scale}}\right)V$$

QKV 表示进一步划分为多个查询头和 K/V 头，每个查询头分配给一个组 [25]。每个 Q 头和 K/V 头对的注意力是并行计算的。由于预填充和解码注意力的资源需求不同，FlashAttention (FA) [29, 30, 49] 和 FlashInfer (FI) [60] 等最先进的库提供了专门的内核 API，针对每个阶段分别进行了优化。这些内核的使用在上下文长度较小的场景中效果很好，其中注意力计算只占总推理时间的一小部分 [24, 62]。

然而，许多现实世界的 LLM 申请中的上下文长度持续增长 [21, 57]。在这种场景中，注意力计算占主导地位，在许多情况下占总推理时间的 60% 以上，如图 4 所示（上下文长度 16K）。请注意，预填充和解码注意力在混合批次中立即依次计算（参见图 3）。因此，当使用独立优化的注意力内核时，GPU 执行会经历资源高需求期，然后是同一资源的低利用率期。例如，预填充内核需要大量计算，但在解码内核执行时计算（大部分）处于空闲状态。

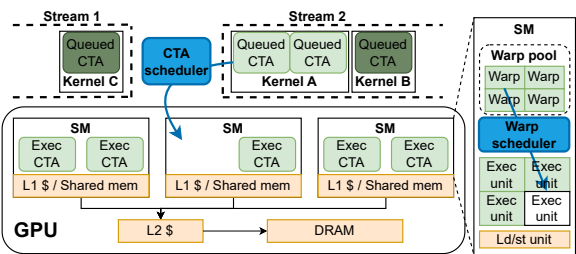


图 5.GPU 执行模型。

我们假设同时计算预填充和解码注意力可以提高性能，因为它会同时利用计算和内存。然而，当前的技术在注意力计算方面存在一些局限性。为了更深入地研究这一点，我们首先解释 GPU 的运行方式，然后介绍在 GPU 上同时执行不同操作的现有方法的案例研究 (§3)。

2.2 GPU执行模型

GPU 的硬件按层次结构排列，支持数十万个并行线程的执行，如图 5 [5] 所示。GPU 的主要处理器单元是流式多处理器 (SM)，现代 GPU 包含大约一百个 SM。每个 SM 都有一个 L1 缓存和共享内存，以及用于加速通用矩阵乘法 (GEMM) 的张量核心和用于整数/浮点运算的执行单元。共享内存是 L1 缓存的用户可寻址分区。SM 通过共享 L2 缓存访问 GPU 内存。

GPU 编程语言公开了模仿硬件层次结构的线程层次结构。最小的执行单元是线程，一组 32 个线程组成一个 warp，它们通常以锁步方式并发执行。为了最大化吞吐量，GPU 程序员确保 warp 中的线程执行相同的代码路径。协作线程阵列 (CTA) [12] 是一组共享 L1 缓存和共享内存的线程束。CTA 中的所有 warp 都保证在单个 SM 内执行。

用户启动 GPU 内核或 GPU 执行的函数，指定 CTA 中的线程数、内核中的 CTA 数量以及每个 CTA 所需的共享内存。然后，此启动会在流中排队；流中的操作是串行的，但不同的流可以以任何顺序并行执行。当 SM 内有足够的执行资源（例如线程、共享内存和寄存器）可用时，CTA 调度程序从流中选择 CTA 并将它们分配给 SM。

GPU 海量吞吐量的核心是硬件中内置的快速、周期级扭曲调度程序。每个时钟周期，warp 调度程序都会调度符合条件的 warp 来执行；如果一个 warp 的线程没有停止（例如，等待内存访问），那么它就是合格的。这使得每个 SM 都能了解上下文

Execution method	GC	WQ	Notes
Streams [45]	×	✓	Easiest to implement
CTA	×	✓	Easy load balancing
Warp (e.g., HFuse [42])	✓	×	Suffers from straggler problem
Intra-thread [53, 59]	✓	×	Cannot overlap with CTA barriers
SM-aware CTA (Ours)	✓	✓	Minimizes operation interference

表 2. 沿着 GPU 执行层次结构的不同级别并发执行或融合不同操作的方法（GC=保证操作共置，WQ=减少波量化）。

Config.	Description
FA_Serial	Serial execution with FA kernels
FA_Streams	Parallel execution via streams with FA kernels
FA_HFuse	Horizontally fused FA kernels with HFuse [42]
POD (Ours)	Optimized fused computation with our kernel

表 3. 混合批次中计算注意力的不同方法（FA: FlashAttention）。

如果需要，可以在每个时钟周期进行切换，从而有效地利用其所有执行资源。

3 并发执行案例

一起计算预填充和解码注意力的最简单方法是将两个输入传递给现有的注意力内核。一些 LLM 服务系统更喜欢这种方法来计算混合批次中的注意力 [7, 17]。在第 5.1 节中，我们表明这会适得其反，并且比串行执行慢。

在本节中，我们重点关注用于并发执行的 GPU 方法，例如并行运行内核或将其操作融合到单个内核中。我们定量分析了它们的性能，并强调了促使我们开发专门的注意力内核的关键局限性。

3.1 并发执行的方法

GPU 中执行层次结构的每个级别都提供了并发执行的潜力（参见表 2）。

1. 内核并行。流可以同时执行不同的 GPU 内核。这种方法很容易实现，因为它只需要将现有的内核提交到不同的流；所有其他方法都需要将不同的操作融合到单个内核中。不幸的是，流本身既不能保证并发性，也不能保证不同操作的 SM 级共置 [45, 63]。2. CTA 并行。在此方案中，内核中的 CTA 以预定方式在操作之间划分。CTA 并行可以实现更好的负载平衡：当一个 CTA 完成执行时，GPU 调度器可以将下一个 CTA 部署到 SM。然而，与流类似，CTA 并行并不保证 SM 级别的共置。3. 扭曲并行。在这里，每个 CTA 内的扭曲被分割到不同的操作中，如水平融合（HFuse

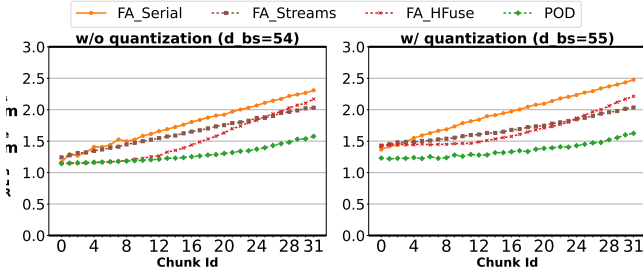


图 6. 32 个混合批次的每层注意力运行时间，对应于 16 K 令牌请求的分块预填充（块大小：512，模型：Yi-6B，d_bs：解码批次大小）。

[42])。此方法保证了共置，因为 CTA 中的所有扭曲都保证驻留在同一个 SM 中。不幸的是，扭曲并行融合遇到了落后问题：整个 CTA 必须完成执行才能被另一个 CTA 取代；如果一个或多个线程或扭曲落后于其他线程或扭曲，则下一个 CTA 会被延迟。在融合预填充和解码注意计算时，融合内核需要进行大量调整以处理不同批次大小和上下文长度的大输入空间，例如，一些混合批次可能预填充繁重，而另一些混合批次可能重解码。因此，融合的预填充解码注意内核特别容易受到扭曲并行融合的散乱效应的影响。

4. 线程内。在线程内融合中，每个线程在不同操作的执行指令之间交替[53, 59]。在简单的情况下，此策略提供了重叠不同操作的最大机会。然而，注意力内核使用 CTA 级同步屏障来协调将数据提取到共享内存中。这些屏障限制了线程内融合作为之前的指令

屏障不能与屏障之后的屏障重叠。我们现在定量分析不同方法的性能。不幸的是，CTA 并行和线程内融合没有现成的实现。因此，我们首先分析注意力内核上的内核并行和扭曲并行方法，然后研究其他方法。

3.2 现有方法分析

对于内核并行执行，如图 6 中的 FA_Streams 所示，我们在两个不同的 CUDA 流上运行 FA 的预填充和解码内核。对于扭曲并行执行 (FA_HFuse)，我们使用[42]提供的工具链融合 FA 的内核。图 6 将它们的性能与 FA 预填充和解码注意内核 (FA_Serial) 的串行执行进行了比较。我们的实验显示了 Yi-6B 对于 16K 提示的 32 个块（块大小 512）的每层注意力计算时间，每个块与每个 16K 上下文长度的解码共同调度。

请注意，如果内核中的 CTA 数量不能被 GPU SM 数量整除，则最后一波调度中的一些 SM 可能会保持空闲状态，这种现象称为

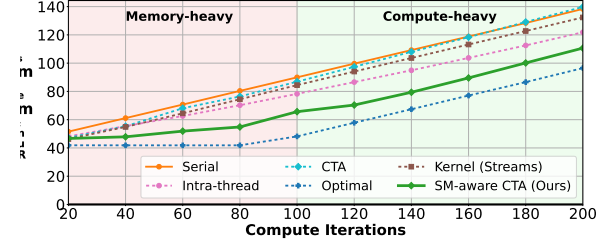


图 7. 细粒度融合与串行计算。

波量化 [38, 44]。在最坏的情况下，由于波量化，工作量的边际增加可能会使内核的延迟加倍。因此，为了充分理解并发执行的好处，我们评估有和没有波量化的性能。在我们的实验中，每个解码请求使用 4 个 CTA（每个 KV 头一个 CTA）。因此，解码批量大小为 54 使用 216 个 CTA，在我们的 NVIDIA A100 GPU（108 个 SM）上没有波量化。相比之下，批量大小为 55 使用 220 个 CTA，留下 4 个量化 CTA。

FA_Streams 比 FA_Serial 提供了一定的速度提升，并且当串行执行受到波量化影响时，其增益更高（高达 20%）。这是因为流并行运行内核来填充 GPU SM，否则这些 GPU SM 会保持空闲状态。这种效果可以在图 6 中看到，其中 FA_Streams 对于两种批量大小所花费的时间大致相同，而 FA_Serial 所花费的时间在批量大小为 55 时会增加；特别是，当批量大小从 54 增加到 55 时，FA_Serial 中的解码时间增加了 25% 以上，这使得预填充和解码的总关注时间增加了 17%。在某些情况下，FA_HFuse 的性能优于 FA_Streams，但由于后面以预填充为主的块中的落后效应，其性能迅速下降。发生这种情况是因为预填充成本随着每个连续块的增加而增加，但所有混合批次中的解码成本相同。总体而言，FA_Streams 和 FA_HFuse 的性能均优于 FA_Serial，但仍具有显著的性能，如 POD-Attention 所示，其性能明显优于这两种方法。

3.3 其他方法分析

对于复杂的内核，例如注意力，有效地实现细粒度的融合方案并不简单，而且容易出错。因此，我们使用一个简单的微基准来分析其他融合方法的性能，该微基准由一个反复将数组元素与标量相乘的计算绑定内核和一个反复将三个数组相加的内存绑定内核组成。每个线程在每次操作后都会执行一个屏障。我们改变计算迭代的次数来评估不同的计算限制和内存限制操作组合下的性能。图 7 显示了应用于这两个函数的不同融合方法的运行时间。在 100 次计算迭代中，两个操作在串行执行时消耗相同的时间。至

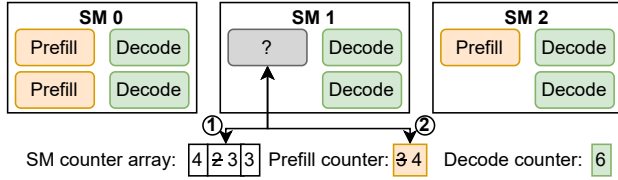


图 8. SM 感知的 CTA 调度。

在这一点左侧，内存限制更为主导。右边是计算限制的。图 7 还显示了理想预言机（即完美重叠）可实现的运行时间。

CTA 和内核并行不能保证计算限制和内存限制操作的 SM 级共置，因此仅比串行执行提供 3% 和 7% 的边际平均改进。线程内融合的性能平均优于串行和 CTA 并行执行 13%。然而，由于同步障碍阻碍了并发执行，线程内融合的好处受到限制。

综上所述，当前并发执行异构操作的方法面临着一些挑战，例如落后者、障碍引起的延迟以及无法保证 SM 级别的共置。在以下部分中，我们将演示专门的融合内核（旨在利用预填充和解码阶段的特征）如何克服这些挑战。

4 POD-注意力

我们引入了 POD-Attention——一个可以有效计算预填充和解码注意力的单一 GPU 内核。我们的主要目标是确保每个 GPU SM 同时计算这两个操作，同时最大限度地减少它们之间的资源争用。我们在 FA v2.6.1 [29] 之上构建内核。

为了实现我们的目标，我们沿着 CTA 维度融合计算，这有助于避免更细粒度的扭曲并行和线程内融合的陷阱。特别是，CTA 并行融合具有三个优点：1) 它允许不同的 CTA 在不同时间独立于其他 CTA 开始和结束，2) 确保同步屏障不会影响计算的其他部分，因为屏障的影响仅限于其 CTA 内，3) 更容易编程（第 4.3 节）。然而，简单的 CTA 并行融合不能保证预填充和解码将在 GPU SM 上共存。为了克服这个限制，我们引入了基于软件的 SM 感知 CTA 调度，其中每个 CTA 在被分派到 SM 后决定是否计算预填充或解码。

4.1 SM感知的CTA调度

SM 感知的 CTA 调度通过“运行时操作绑定”将预填充和解码 CTA 放在一起。在这里，CTA 在检查以下内容后决定是否在运行时执行预填充或解码：1) 它在哪个 SM 上启动[56]，以及 2) 在同一 SM 上运行的其他 CTA 正在做什么。这

```

1  if (threadIdx.x == 0) { // Leader thread finds assignment
2      int sm_id; // Find which SM this CTA is on
3      asm volatile("mov.u32 %0, %%smid;" : "=r"(sm_id));
4      // For this SM, what do we want to run?
5      const int ratio = (prefill_ratio + decode_ratio);
6      int op, ticket = (atomicAdd(&sm_ctr[sm_id], 1) % ratio);
7      if (ticket < prefill_ratio) op = PREFILL;
8      else op = DECODE;
9      // Get the next CTA for operation
10     int cta_id = atomicAdd(&cta_assign[op], 1);
11     // If the CTA exceeds the max CTA for that op switch ops
12     if (op == PREFILL && cta_id >= prefill_ctas) {
13         op = DECODE;
14         cta_id = atomicAdd(&cta_assign[op], 1);
15     } else if (op == DECODE && cta_id >= decode_ctas) {
16         op = PREFILL;
17         cta_id = atomicAdd(&cta_assign[op], 1);
18     }
19     // Write the CTA ID and operation to shared memory
20     shared_mem[0] = cta_id;
21     shared_mem[1] = op;
22 }
23 __syncthreads(); // Barrier: waits for scheduling to finish
24 // Fetch the assigned CTA and operation.
25 int cta_id = shared_mem[0];
26 const int op = shared_mem[1];
27 __syncthreads();
28 // Perform the appropriate operation
29 if (op == PREFILL) prefill_op(cta_id);
30 else decode_op(cta_id);

```

图 9. 用于 SM 感知 CTA 调度的 CUDA 代码。

允许内核完全不知道硬件调度程序如何将 SM 分配给 CTA。

为此，在启动内核之前，我们确定独立预填充和解码所需的 CTA 数量，并使用与两者总和匹配的 CTA 启动内核。每个 SM 都有一个计数器，用于跟踪在其上启动的 CTA 数量，还有另外 2 个计数器，用于跟踪迄今为止在其上执行的预填充和解码 CTA 的数量。

图 9 显示了 SM 感知的 CTA 调度的简单代码片段。当硬件调度程序在 SM 上调度新的 CTA 时，CTA 的领导线程（例如线程 0）会读取 SMID 硬件计数器 [13]，其中包含启动它的 SM 的唯一 ID（第 2 - 3 行）。然后该线程对 SM 计数器执行原子添加操作以获得票证（第 6 行）。该票证通知线程应执行哪个操作，即预填充或解码（第 7 - 8 行），具体取决于调度策略。该线程还增加操作的 CTA 计数器（第 10 行）。如果这超出了该操作的最大 CTA，则会切换操作（第 12 - 18 行）。最后，它将这信息写入共享内存，以便 CTA 中的其他线程可以相应地开始执行（第 20 - 30 行）。我们研究了两种调度策略：50:50 和比例。在 50:50 策略中，SM 上的后续 CTA 在预填充和解码之间交替。相反，比例策略（第 5 行）根据当前批次中预填充和解码 CTA 的比率来分配 CTA。

4.2 性能优化

简单地将预填充和解码操作放在一起并不能产生最佳性能。在本小节中，我们介绍

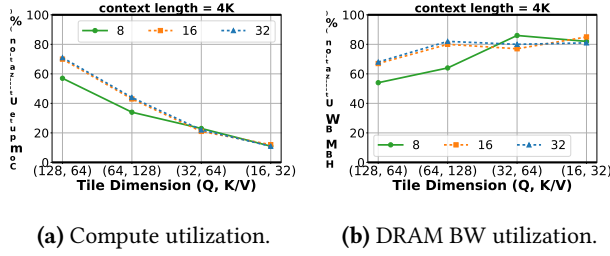


图 10. 批量大小为 8、16 和 32 时，解码切片大小对计算和 HBM BW 利用率的影响。

各种优化，以最大限度地发挥融合预填充和解码注意力计算的优势。

4.2.1 瓷砖尺寸。数据分块对于有效利用张量核心是必要的，张量核心提供的吞吐量比对应的 CUDA 核心高 $\sim 8\times$ [26]。平铺还有助于提高共享内存的使用率。然而，平铺的好处在不同的操作中并不统一。解码对每个请求的单个令牌进行操作，在查询序列长度 (QSL) 维度上的瓦片长度为 1。在 Group Query Attention [25] 中，这个长度增加到查询头和 KV 头之间的比率，通常为 2–8。由于维度长度很小，数据重用微不足道，并且性能受到内存带宽的限制。

FlashAttention 使用 64–128 的图块长度作为 QSL 维度。使用如此大的图块大小的副作用是解码最终会被零填充，从而导致冗余计算[34]。例如，图10a显示解码注意力内核的计算利用率与图块大小成正比，在QSL图块尺寸为128时达到70%，而图块尺寸为16时为10%。但是，请注意，解码注意力是内存限制的，因此，解码内核的主要目标是尝试使内存带宽饱和。图 10b 显示，即使 QSL 切片尺寸相对较大 (64)，解码内核也能够最大化内存带宽利用率。因此，对于仅解码注意内核，没有什么动机进一步减小图块大小。

相比之下，在融合内核中使用大块大小进行解码会适得其反：解码执行的任何冗余计算都会干扰共置预填充，因为张量核心在它们之间共享。如果我们减少不必要的计算，预填充可以更好地利用张量核心。为此，我们对 QSL 使用 16 的解码瓦片长度，这是 CUTLASS [11] A100 张量运算所需的最小值。这会将解码的计算利用率降低至 $\sim 10\%$ ，从而释放张量核心以进行预填充。图 10b 显示，减小图块大小不会对大批量大小的解码性能产生不利影响。

4.2.2 每个 SM 的并发 CTA。 SM 上同时运行的 CTA 数量决定了每个 CTA 可以拥有的资源量（例如共享内存）。每个 SM 更多的 CTA 意味着每个 CTA 的资源更少，但细粒度调度和共置的机会更多，即，每个 SM 有 2 个 CTA，我们只能以 1:1 的比例共置预填充和解码，但每个 SM 有 4 个 CTA，我们可以根据批次组成以不同的比例分配 CTA 来预填充和解码，例如，3 个 CTA 用于预填充，1 个 CTA 用于解码。一般来说，预填充受益于每个 SM 更少的 CTA，因为它允许每个 CTA 访问更多共享内存，从而能够使用更大的图块大小。相比之下，解码不会从较大的图块大小中受益，因此每个 SM 使用更多的 CTA 可能是有益的，因为它允许细粒度的调度。

为了实现两全其美，POD-Attention 支持两种配置：对于预填充主导的混合批次，每个 SM 2 个 CTA，否则每个 SM 4 个 CTA。根据所需的配置，我们修改用于预填充和解码的图块长度和线程数。我们还探讨了每个 SM 8 个 CTA 是否可以进一步提高性能，发现它只能在少数情况下略微提高性能，而在大多数情况下表现不佳。POD-Attention 在运行时自动选择最合适的配置。

4.2.3 虚拟解码 CTA。融合内核中提供给每个预填充和解码 CTA 的共享内存量必须相同。然而，由于解码使用较小的图块大小，因此解码的共享内存要求是预填充要求的四分之一。为了避免为解码过度分配共享内存，我们将每个解码 CTA 划分为包含线程扭曲的虚拟 CTA。如果原始解码 CTA 有四个 warp，则每个虚拟 CTA 包含一个 warp，该 warp 使用原始 CTA 的四分之一的共享内存。每个常规 CTA 中所有虚拟 CTA 使用的共享内存总和接近预填充使用的共享内存。这样，虚拟解码 CTA 可以平衡预填充和解码使用的共享内存。

4.2.4 限制预填充分割。FlashAttention 跨查询头和 QSL 平铺维度并行计算。FlashDecoding [31] 专为 QSL 为 1 的解码而设计，当没有足够的并行度来填充 GPU 的 SM 时，它会进一步在 K/V 维度上分割计算。这种方法的副作用是，不同的 CTA 彼此独立地从内存中获取相同的查询张量，与分割数量成正比。因此，分割计算会增加内存带宽利用率。当输入包含足够的令牌时，预填充不需要沿着键/值维度进行分割，但分块预填充通过设计限制了每次迭代处理的令牌数量（以最小化 TBT）。因此，FlashAttention 还使用 FlashDecoding 技术来加速分块预填充注意力计算。该方案非常适合仅预填充内核，因为增加的并行性可以轻松抵消额外内存读取的成本。

然而，在融合内核中，使用大量分割进行分块预填充可能会导致预填充和解码 CTA 之间出现内存带宽竞争，从而可能抵消融合的优势。为了平衡这种权衡，我们限制分块预填充的分割数量，以填充最多两个完整的波（根据经验确定）。这允许分块预填充在需要时使用更多 CTA，同时确保拆分数目不会过多并损害并发解码。

4.3 实现CTA并行融合

为了融合两个内核，我们首先将它们转换为可从 GPU 代码内调用的通用设备函数，同时删除对 CUDA 提供的 CTA ID（即 blockIdx）的所有引用，而是将其作为函数参数传递。我们构建一个包装器内核，使用计算出的 CTA ID 调用这些不同的函数。预填充和解码操作的执行就像提供的 CTA ID 是它们的实际 ID 一样。这使得能够灵活地重新映射 CTA ID，例如，融合内核的 CTA 0 可以使用 CTA ID 0 调用预填充，CTA 1 可以使用 ID 0 调用解码，CTA 2 可以使用 ID 1 调用预填充，依此类推。每个 CTA 获得的共享内存量在内核启动时是固定的，预填充和解码操作有不同的要求。为了解决这个问题，我们手动调整预填充和解码操作的共享内存使用情况，以平衡它们的需求，同时最大限度地减少性能下降。我们使用足够的共享内存启动融合内核，以满足任一操作所需的最大内存。为了实现虚拟 CTA，我们修改了解码函数，用扭曲级屏障替换所有 CTA 级屏障。使用适当的虚拟 CTA ID（而不是分配的 CTA ID）调用融合内核中的解码函数。

4.4 替代实现的讨论

并发执行是 GPU 文献中一个经过深入研究的主题 [40,43,55,61]，我们可以通过多种方式实现重叠预填充和解码注意力计算的高级目标。一种值得注意的策略是基于持久线程 [32,45,63]：在这种方法中，启动预定数量的 CTA（足以完美填充所有 SM）。这些 CTA 的持久线程根据需要拉动正确类型的工作（例如，预填充或解码图块）。我们发现这种策略也缓解了掉队问题。然而，仍然需要 SM 感知调度来决定在哪个持久 CTA 上运行哪些工作（预填充或解码），这对于保证 SM 内的操作共置至关重要。将其与 SM 感知调度集成后，我们发现该策略的性能与我们的 CTA 并行融合相当。

NVIDIA 还提供 MPS（多进程服务）[10] 和 MIG（多实例 GPU）[9] 功能，以便在同一 GPU 上并行运行不同的应用程序。然而，由于混合批处理在设计上将预填充和解码操作结合在一个进程中，MPS 和 MIG 不适用于我们的用例。

Model	GPU	# Q Heads	# KV Heads	# Layers
Yi-6B	1 A100	32	4	32
Llama-2-7B	2 A100s	32	32	32
Llama-3-8B	2 A100s	32	8	32

表 4. 用于评估的模型和硬件。

5 评价

我们的评估回答了以下问题：

- POD-Attention 对注意力计算延迟有何影响？
- POD-Attention 如何影响端到端 LLM 推理性能？
- POD-Attention 中采用的不同优化和设计选择有何影响？

模型和环境：我们使用 Yi-6B（4 KV 头 [20]）、Llama-2-7B（32 KV 头 [6]）和 Llama-3-8B（8 KV 头 [8]）评估 POD-Attention，在一个 A100 GPU 上部署 Yi-6B，在两个具有张量并行性的 A100 GPU 上部署 Yi-6B（表 4）。每个模型有 32 个查询头。每个 GPU 具有 80GB HBM 内存。

工作负载和指标：我们评估离线和在线推理场景。对于离线推理，我们报告每分钟处理的请求数。对于在线推理，我们报告了两个工作负载的 TTFT、TBT 和请求执行延迟，每个工作负载包含 2K 请求，每个请求的上下文长度范围为 4K 到 32K 令牌。其中一个工作负载是内部企业工作负载（平均上下文长度为 10.5K 令牌，每个请求预填充解码令牌比率，即 P:D 在 0–40 范围内），另一个工作负载基于 arXiv-Summarization [4]（平均上下文长度为 9.5K 令牌，P:D 比率为 0-50）。平均而言，arXiv 工作负载中的解码令牌数量 (470) 比内部工作负载 (331) 高 42%。

服务系统基线：我们的实验使用 SarathiServe [15] 作为服务框架，它构建在 vLLM [19] 之上。我们评估两个基线：1) 原始 vLLM 调度程序 [41]，它在单独的批次中运行预填充和解码，优先考虑预填充而不是解码；2) Sarathi-Serve [23]。两个基线都使用 FlashAttention 内核 (v2.6.1) 进行注意力计算。我们将 POD-Attention 集成到 Sarathi-Serve 中，以评估我们优化的好处。为简单起见，我们将不带 POD-Attention 的 Sarathi-Serve 称为 Sarathi 和 Sarathi+POD。

5.1 评估注意力计算

图 6 展示了 POD-Attention 加速注意力计算的特定实例，其性能比次佳替代方案高出 29%。为了证明 POD-Attention 的广泛适用性，我们对模型上的一千多个混合批次进行了全面扫描。在这些实验中，我们将上下文长度从 4K 更改为 20K，将预填充块大小从 512 更改为 2K。我们专注

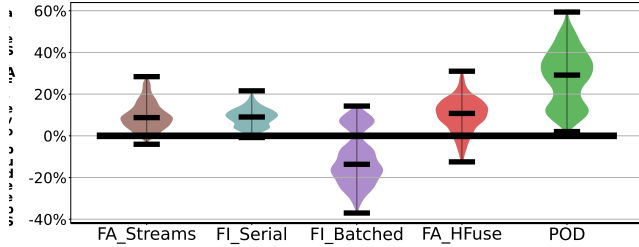


图 11. 与 FA_Serial 相比，不同机制的注意力计算加速分布。

预填充和解码注意力至少占串行运行时间 20% 的场景，因为其他情况通过操作融合提供的优化潜力有限。

除了 FlashAttention 内核之外，我们还比较了两种配置下 FlashInfer (FI) v0.2.0 内核 [60] 的运行时间：FI_Serial 和 FI_Batched。FI_Batched 使用 Flash-Infer 的预填充内核计算预填充和解码注意力。我们与 FI_Batched 进行比较有两个原因：1) 这种策略是同时计算预填充和解码注意力的最简单方法，2) 一些系统更喜欢这种方法，例如 Sarathi 在其默认注意力后端 [7] 中使用 FI_Batched，并且 vLLM [17] 中要求类似的功能。然而，我们表明这种策略效率低下，例如，当 FI_Batched 使用预填充优化内核时，由于使用较大的图块大小，它会导致解码计算中出现冗余计算（第 4.2.1 节）。这种冗余计算会干扰共同运行的预填充。如果 FI_Batched 使用解码优化内核，内存带宽上也会出现类似的干扰。

图 11 显示了与 FA_Serial 相比不同机制的相对加速。FA_Streams 提供的加速有限，因为它不能保证 SM 级别的操作重叠。在极少数情况下，我们发现流同步的开销也会抵消它的好处。FI_Serial 具有更好优化的解码内核，相对 FA_Serial 有一定的改进，但它不重叠操作。FI_Batched 提高了低上下文长度下的性能，但由于解码的冗余计算，在较高长度下性能下降高达 40%。FA_HFuse 是最强的基线，因为它保证了操作重叠，将中值性能提高了 11%。然而，FA_HFuse 容易受到散乱效应的影响，因此与 FA_Serial 相比，其速度慢达 13%。从图 6 中还可以看到后面的块的散乱效应，其中预填充占主导地位，因此很难实现完美的利用率。

POD-Attention 的峰值加速率为 59%，平均加速率为 28%——高于所有替代方案。我们发现，在 25% 的情况下，它也达到了理论峰值加速的 10% 以内，这意味着近乎完美的重叠。此外，与其他替代方案不同，POD-Attention 的串行执行性能绝不会低于预期。这些结果强调了专门的注意力内核对于基于混合批处理的 LLM 推理的重要性。

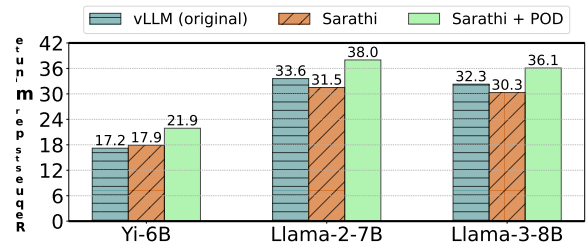


图 12. 离线推理中的服务吞吐量。

此外，我们还分析了注意力内核的能耗，并观察到 POD-Attention 比 FA_Serial 减少了高达 35% 的能耗（平均减少了 20.5%）。这些节省很大程度上与运行时间的减少成正比，这表明预填充解码重叠不仅提高了性能，而且还降低了能耗。

5.2 评估离线推理的吞吐量

为了评估离线推理场景，我们运行每个 16K 令牌的长上下文请求。我们对 Yi-6B 使用块大小 512，对 Llama-2-7B 和 Llama-3-8B 使用 1K，选择的方式是对提示进行分块不会降低线性操作的性能（如 Sarathi [23, 24] 所建议的）。我们对 Yi-6B 总共运行 1K 个请求，对 Llama-2-7B 和 Llama-3-8B 各运行 2K 个请求，这样单个配置的总运行时间约为一小时。每个请求的输出令牌数量对于 Yi-6B 设置为 2K，对于 Llama-3-8B 设置为 1K，对于 Llama-2-7B 设置为 256；我们在第 5.4.4 节中研究了不同预填充对解码令牌比率（P:D 比率）的影响。

图 12 显示 Sarathi+POD 通过以下方式提供最佳服务：-put：三个模型比 Sarathi 高 22%、20% 和 19%，比 vLLM 高 27%、13% 和 12%。值得强调的是，分块预填充和混合批处理需要权衡。由于重复的 KV 缓存加载，对提示进行分块会增加注意力计算时间：计算预填充块的注意力需要读取所有先前块的 KV 缓存 [65]。同时，将解码令牌与预填充融合有助于更有效地执行线性操作：不需要单独读取预填充和解码的模型权重。因此，vLLM 和 Sarathi 的相对性能可能会因工作负载、模型配置和块大小而异。在我们的实验中，Sarathi 比 Yi-6B 的 vLLM 略微提高了吞吐量，但对于 Llama-2-7B 和 Llama-3-8B 的吞吐量却较差。Sarathi+POD 在所有操作中融合预填充和解码，以提高 GPU 资源利用率，从而优于两个基线。

5.3 评估在线推理的延迟

我们在接近系统服务容量的内部和基于 arXiv 的工作负载上评估 Llama-3-8B：系统在避免高排队延迟的同时可以处理的最大负载 [23]。我们通过以下方式评估每个工作负载中的 2048 个请求：

QPS	System	TTFT		TBT		Request Latency		% Requests with Stalls	
		P50	P99	P50	P99	P50	P99	200ms	500ms
1.1	vLLM (original)	0.67	10.11	0.04	1.13	25.05	91.01	99.95	97.8
	Sarathi	2.2	12.58	0.10	0.15	26.83	92.24	2.05	0
	Sarathi+POD	1.9	12.26	0.10	0.14	24.70	79.04	3.17	0
1.2	vLLM (original)	0.94	12.70	0.07	1.76	42.73	151.8	99.95	99.6
	Sarathi	25.44	57.83	0.12	0.16	67.12	140.5	5.07	2.63
	Sarathi+POD	7.49	23.78	0.11	0.15	38.69	106.8	2.29	0

表 5. 内部工作负载。延迟数（以秒为单位）。

QPS	System	TTFT		TBT		Request Latency		% Requests with Stalls	
		P50	P99	P50	P99	P50	P99	200ms	500ms
0.85	vLLM (original)	0.55	6.26	0.03	0.82	20.53	234.93	99.9	97.8
	Sarathi	2.68	14.89	0.08	0.13	27.87	281.07	4.15	2.05
	Sarathi+POD	1.85	12.71	0.08	0.11	24.31	255.75	1.85	1.61
0.95	vLLM	0.71	8.25	0.06	1.36	36.86	401.2	99.9	99.45
	Sarathi	46.22	144.2	0.1	0.14	90.12	417.6	4.44	1.9
	Sarathi+POD	11.74	27.38	0.09	0.12	40.6	333.0	2.2	2.1

表 6. 基于 arXiv 的工作负载。延迟数（以秒为单位）。

根据泊松分布改变输入负载。对于 Sarathi 和 Sarathi+POD，我们对基于 arXiv 的工作负载使用块大小 1024，对预填充量更大的内部工作负载使用 1536。我们讨论 TTFT、TBT 和端到端请求执行延迟等重要的 LLM 特定延迟指标的性能。

请注意，这些指标之间存在固有的权衡[23]，并且优化一个指标可能会严重损害其他指标。例如，如下所示，vLLM 优先考虑预填充，从而实现低 TTFT，但牺牲了 TBT，导致 95+% 的用户请求在解码生成期间经历一个或多个停顿。另一方面，与 vLLM 相比，Sarathi 将停顿减少到用户请求的一小部分，但显著增加了 TTFT。

5.3.1 薄膜晶体管。vLLM 提供最低的 TTFT，因为它会在第一个可用机会时安排预填。相比之下，Sarathi 增加了 TTFT，因为正在进行的解码会干扰预填充。Sarathi 中的 TTFT 随着负载的增加而进一步增加，特别是由于更高的排队延迟，例如，内部和基于 arXiv 的工作负载的 TTFT 中位数分别为 25.4 和 46.2 秒，而 vLLM 的 TTFT 为 0.94 和 0.71 秒。与 Sarathi 相比，Sarathi+POD 显著降低了 TTFT，在较高负载下将 TTFT 中值分别降至 7.5 和 11.74 秒。Sarathi+POD 还比 Sarathi 将 P99 TTFT 减少了多达 4.3×。

5.3.2 TBT 和失速。每当安排新的预填充时，vLLM 就会暂停正在进行的解码，从而导致生成停顿，从而导致与 LLM 服务的交互性较差。这些生成停滞反映为高尾 TBT 延迟，例如，vLLM 的 P99 TBT 达到 1.76 秒（内部工作负载）和 1.36 秒（基于 arXiv 的工作负载）。

Latency Metric	vLLM (original)	Sarathi+POD		
		1024	1536	2048
TTFT (P50)	0.67	6.29	1.9	1.59
TTFT (P99)	10.11	18.99	12.26	12.40
TBT (P50)	0.04	0.08	0.10	0.08
TBT (P99)	1.13	0.11	0.14	0.18

表 7. 具有不同块大小的 Sarathi+POD 与 vLLM 的 TTFT 和 TBT（内部工作负载，QPS 1.1）。

在最坏的情况下，我们观察到，当 vLLM 连续计算多个预填充时，TBT 延迟最高可达 8 秒。相比之下，Sarathi 可确保正在进行的解码不会受到新预填充的影响。因此，与 vLLM 相比，Sarathi 提供了显着更低的尾部 TBT 延迟，例如，Sarathi 的 P99 TBT 最多为 0.16 秒（比 vLLM 低 10×）。Sarathi+POD 进一步将尾部 TBT 比 Sarathi 减少了 10 – 20%。至关重要的是，由于单个响应会导致大量解码，因此高 TBT 尾部延迟会影响 vLLM 中的几乎所有请求，这意味着几乎所有用户的交互体验都很差。即使 TBT SLO 提高到 500 毫秒，超过 97% 的总请求至少会在 vLLM 中经历一次停顿。相比之下，很少有请求 (<5%) 观察到 Sarathi 中的停顿，而在大多数情况下，Sarathi+POD 进一步减少了停顿。

5.3.3 端到端请求延迟。请求延迟可用于估计在线推理中的系统吞吐量。对于 QPS 1.2 的内部工作负载，Sarathi 比 vLLM 的 P99 请求延迟减少了 8%，但对于基于 arXiv 的工作负载 (QPS 0.85)，它比 vLLM 增加了 24%。Sarathi+POD 不仅在所有情况下都优于 Sarathi，而且

CL	Batch size					CL	Batch size				
	8	16	32	64	128		8	16	32	64	128
1024	1.08	1.00	1.07	1.14	1.03	1024	1.00	1.00	1.00	1.00	1.00
2048	1.00	1.00	1.00	1.09	1.05	2048	1.10	1.06	1.05	1.00	1.00
4096	1.00	1.00	1.00	1.00	1.00	4096	1.13	1.12	1.11	1.05	1.04
8192	1.00	1.00	1.00	1.00	1.00	8192	1.13	1.15	1.16	1.11	1.08
16384	1.00	1.00	1.00	1.00	1.00	16384	1.17	1.16	1.17	1.17	1.15

(a) 2 CTAs per SM.

(b) 4 CTAs per SM.

图 13. 具有不同 CTA 配置的 POD-Attention。

在许多情况下，它的性能也优于 vLLM，例如，对于内部工作负载，它比 vLLM 减少了高达 42% 的 P99 请求执行延迟（106.8 秒 vs 151.8 秒，QPS 1.2）；对于基于 arXiv 的工作负载，它的 P99 请求执行延迟减少了高达 17%（333 秒 vs 401.2 秒，QPS 0.95）。

这些结果表明，Sarathi 通过减少尾部 TBT 和最大限度地减少停顿来增强交互性，尽管与 vLLM 相比，TTFT 有所增加，吞吐量有所降低。POD-Attention 优化了 Sarathi 所有指标的性能，有效平衡了吞吐量与延迟之间的权衡。表 7 显示，Sarathi+POD 中的块大小可以进一步调整，以实现 TTFT 和 TBT 的权衡，例如，使用较大的 2K 令牌块大小可以将 TTFT 中值从 6.3 秒降低到 1.6 秒，但代价是更高的 TBT（P99 0.18 秒 vs 0.11 秒）。

5.4 敏感性研究

5.4.1 每个 SM 的 CTA。图 13 显示了 Llama-3-8B 上同时运行不同数量的 CTA、不同批量大小（水平）和上下文长度（垂直）的 POD-Attention 性能。对于每个（上下文长度、批量大小）数据点，我们将运行时标准化为两种配置中的最佳配置。一般来说，对于预填充成本占主导地位的长上下文，每个 SM 2 个 CTA 性能更好，因为它允许更大的图块尺寸。随着上下文长度的减小，解码成本开始显现，因此每个 SM 4 个 CTA 开始表现更好：每个 SM 更多的 CTA 允许用更少的预填充打包更多的解码，例如 1 个预填充 CTA 和 3 个解码 CTA。

5.4.2 调度策略。我们探索 SM 中的两种 CTA 调度策略，即 50:50 分配和比例分配。在 50:50 分配中，CTA 在预填充和解码之间交替的 SM 上启动，即第一个 CTA 执行预填充，下一个执行解码，依此类推。该策略与内核中预填充和解码 CTA 的总数无关。在比例分配中，CTA 根据内核中 CTA 的总数来选择是否执行预填充或解码。例如，如果需要 50 个预填充和 100 个解码 CTA，则每个 SM 上的第一个 CTA 将执行预填充，接下来的两个 CTA 将执行解码，然后重复。图 14 显示了在 Yi-6B 和 Llama-3-8B 上使用这些 8K 上下文长度和不同解码批量大小的策略的 POD-Attention 的延迟。我们注意到

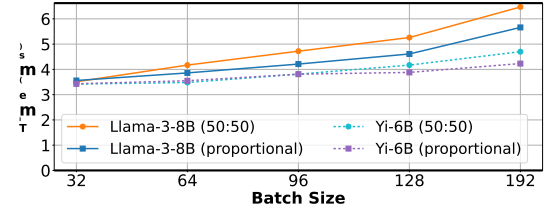


图 14. POD-Attention 中调度策略的效果。

Chunk Id	FA_Serial	POD-ATTENTION	
		Vanilla split	Limited split [Ours]
28	1.93	1.68 (0.87×)	1.45 (0.75×)
29	1.96	1.69 (0.86×)	1.45 (0.74×)
30	1.98	1.71 (0.86×)	1.45 (0.73×)
31	1.99	1.71 (0.86×)	1.46 (0.73×)

表 8. 提示的最后四个预填充块的每层注意力运行时间（毫秒），与解码批量大小 64 共同运行（模型：Llama-3-8B，上下文长度：16K，块大小：512）。

随着负载增加（更大的批量大小），比例分配的性能会提高超过 50:50 分配。按比例分配可分散不太频繁的操作，从而实现更好的操作重叠并减少资源争用，比 50:50 分配方案的性能提高高达 14%。

5.4.3 限制预填充分割。POD-Attention 通过默认的 Flash Decoding 样式沿 KV 维度分割来减少注意力计算时间。然而，限制分割数量可以进一步提高性能。例如，表 8 显示，在 16K 提示的最后四个块中，与相同上下文长度的 64 个解码请求共同运行，限制预填充注意力计算中的分割数量几乎使 POD-Attention 相对于 FA_Serial 的加速增加了一倍。

5.4.4 对工作负载的敏感性。POD-Attention 加速了混合批次的执行，因此它对整体性能的影响取决于给定工作负载中由混合批次组成的迭代次数。主要由预填充（高 P:D 比率）或解码（低 P:D 比率）主导的工作负载可能不会从 POD-Attention 中获得什么好处。为了了解不同 P:D 比率的影响，我们使用总共 2048 个请求对 Llama-3-8B 进行基准测试，每个请求包含 $\approx 16.5K$ 令牌，但具有不同的 P:D 比率（在 8 到 24 的范围内），例如，如果 P:D 为 10，则请求包含 $\approx 15K$ 预填充令牌和 $\approx 1.5K$ 解码令牌。图 15 显示，Sarathi+POD 在不同的工作负载组合中均优于 Sarathi。峰值增益出现在 12 至 18 的 P:D 范围内，因为在此状态下大多数批次都是混合批次。相反，当 P:D 比率低于 12 时，许多迭代运行仅解码批次（或者当 P:D 比率高于 18 时，运行仅预填充批次）。

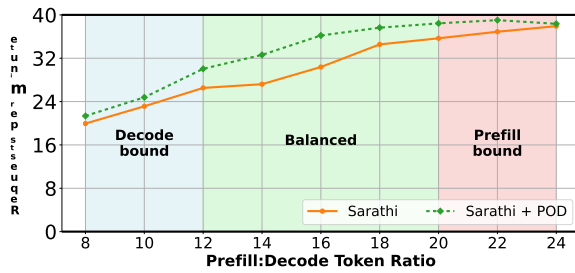


图 15. 不同工作负载分布下的请求处理吞吐量（型号：Llama-3-8B、TP-2）。

6 相关工作

优化 GPU 执行和 LLM 服务系统是一个活跃的研究领域 [21, 23, 27, 33, 35, 36, 38, 39, 41, 46–48, 52, 57, 62, 64–66]。

优化注意力计算：FlashAttention [30] 引入了第一个专门的注意力实现，通过基于图块的计算将其所有操作融合到单个内核中。FA-2 [29] 通过更好的工作分区和负载平衡进一步改进了它。FlashDecoding [31] 通过沿 KV 维度分割计算来加速解码注意力。FlashDecoding++ [34] 使用异步 softmax、双缓冲平面 GEMM 优化和基于数据流的硬件资源自适应来加速解码。LeanAttention [48] 遵循平铺计算的 Stream-K 缩减 [44]，以实现在 SM 之间更好的负载分配以进行解码。FlashInfer [60] 引入了基于共享前缀的优化注意力内核。与单独处理预填充和解码的工作相比，POD-Attention 将它们联合优化并融合到单个内核中。

FA-3 [49] 是 FlashAttention 内核系列的最新成员。它利用 NVIDIA Hopper 架构中的新功能，利用张量核心、张量内存加速器和特殊功能单元的异步性。在撰写本文时，FA-3 正在积极开发中，因此我们将 POD-Attention 支持扩展到 FA-3 和 Hopper 架构以供将来的工作。

运算融合：内核融合是提高 GPU 性能的常用技术。Elastic kernels [45] 建议限制资源以允许同时运行多个内核。然而，该方法不能保证 SM 内共置。为了克服这个问题，ISPA [63] 为每个内核部署了预定数量的 CTA，少于 GPU 上同时运行的 CTA 数量。使用重要的先验分析来确定适当的 CTA 大小，以允许两个内核同时执行。对于具有动态变化输入大小的注意力内核来说，这可能是乏味的，并且使得预填充和解码操作之间的负载平衡变得困难，因为

操作提前完成导致资源未得到充分利用。HFuse [42] 以扭曲并行方式融合操作，提供源到源编译工具来融合内核。以 SM 为中心的调度 [56] 使用 SM 计数器将工作分配给 CTA，我们在 POD-Attention 中利用了这一点。

优化 LLM 推理：优化 LLM 服务系统是一个活跃的研究领域 [23, 33, 36, 41, 46, 50–52, 57, 58, 62]。Orca [62] 引入了迭代级调度，以消除不同长度的请求批量在一起时的计算碎片。PagedAttention [41] 和 vAttention [47] 提出了用于 LLM 推理的动态内存管理的不同技术。Sarathi-Serve [23] 利用分块预填充来实现无停顿批处理。相比之下，Splitwise [46]、DistServe [65] 和 TetriInfer [35] 将预填充和解码阶段分散到不同的 GPU 节点上，以避免这些阶段之间的干扰。最近的各种工作还提出了将计算与通信重叠以提高资源利用率 [28, 37, 54]。

与 POD-Attention 类似，NanoFlow [66] 也致力于提高设备内资源利用率，尽管采用了相反的方法。NanoFlow 将批次划分为更小的操作级纳米批次，并通过 CUDA 流以与互补资源配置文件重叠操作的方式对其进行调度。相比之下，POD-Attention 试图通过融合预填充和解码注意力计算来最大化给定批次内的资源利用率。虽然 NanoFlow 需要大批量才能从批量分割中受益，但当注意力消耗大量时间时，POD-Attention 非常有用。因此，NanoFlow 似乎更适合小上下文场景，而 POD-Attention 则针对长上下文场景，这些场景依赖于混合批处理来实现高效的 LLM 服务。

7 结论

我们引入了 POD-Attention——第一个专门用于并行计算预填充和解码注意力的注意力内核，以便可以同时利用 GPU 的计算和内存带宽。与使用独立优化的预填充和解码注意力内核相比，POD-Attention 可将注意力计算加速高达 59%（平均 28%），从而实现基于 LLM 推理的高效混合批处理。POD-Attention 还将端到端服务吞吐量提高了 22%，同时显著降低了最先进的 LLM 服务系统 Sarathi-Serve 和 vLLM 的延迟。

致谢

我们感谢我们的牧羊人 Tim Rogers、匿名 ASPLOS 审稿人以及 Ajay Nayak 对本文各个方面的宝贵反馈。我们还感谢 Zihao Ye 对 POD-Attention 和 FlashInfer 进行的各种有益的讨论。Aditya K Kamath 和 Simon Peter 得到国家自然科学基金会拨款 CNS-2212580 的支持。

神器附录

A.1 摘要

POD-Attention 是一个 GPU 内核，它重叠了大型语言模型的预填充和解码注意操作。POD-Attention 构建在 FlashAttention 内核 (v2.6.1) [29] 之上，并与 Sarathi-Serve [23] 集成——一种最先进的基于混合批处理的 LLM 推理调度程序。

A.2 工件检查表（元信息）

- 编译：CUDA 12.4, GCC 11.4。
- 型号：Llama-2-7B [6]、Llama-3-8B [8]、Yi-6B [20]。
- 数据集：arXiv-Summarization [4]。
- 运行时环境：Ubuntu 22.04、CUDA 12.4、Python 3.12 和 PyTorch 2.4。
- 硬件：1-2 个 NVIDIA A100 80 GB GPU、x86 机器。
- 准备工作流程需要多长时间？：使用 Docker 映像 1 分钟。如果从源安装，则需要 1-2 小时。
- 完成实验需要多少时间（大约）？：大约。18小时。

- 公开吗？：是的。
- 已存档（提供 DOI）？：10.5281/zenodo.14770841

A.3 描述

A.3.1 如何访问。我们以各种形式提供源代码：Docker 容器（参见A.3.3）、GitHub存储库（https://github.com/microsoft/vattention/tree/main/pod_attn）和Zenodo（<https://doi.org/10.5281/zenodo.14770840>）。

A.3.2 硬件依赖性。此工件需要一台 x86 机器，配备 2 个 NVIDIA A100 GPU，每个 GPU 80GB。如果只有一个 GPU 可用，则除表 6 以及图 12 中 Llama-2-7B 和 Llama-3-8B 的结果外，所有实验都可以完整进行。

A.3.3 软件依赖性。POD-Attention 已在 Ubuntu 22.04 的机器上进行了测试。所有其他软件依赖项均在安装时解决。

A.3.4 数据集。一些实验基于 arXiv-Summarization 数据集。我们使用工件的 Traces/ 文件夹中可用的数据集的子集。

A.3.5 模型。该工件评估 Yi-6B、Llama-2-7B 和 Llama-3-8B。访问 Yi-6B 和 Llama-2-7B 很简单，但访问 Llama-3-8B 需要使用用户的私有令牌（下面的 HF_TOKEN）登录 Huggingface：

```
$ Huggingface-cli 登录 --token HF_TOKEN
```

A.4 安装

我们提供两种安装和测试方法：使用Docker（推荐）或手动安装。

A.4.1 Docker 安装（推荐）。我们为 POD-Attention 提供了一个 docker 镜像，并预安装了所有依赖项。您可以启动 docker 容器并导航到工件目录，如下所示：

```
$ docker run --gpus all -it \ -p 8181:8181 --rm --ipc=host --cap-add=SYS_ADMIN \ mnp1910/pod_attention:asplos_25_pytorch_r \ un $ cd /workspace/vattention/pod_attn
```

A.4.2 手动安装。对于手动安装，我们可以将 POD-Attention（在 vAttention 存储库中提供）下载到主目录进行安装。我们将 Anaconda 用于相应版本的 CUDA、Python 和 PyTorch。这最多可能需要 2 小时。

```
$ git clone \ https://github.com/microsoft/vattention.git $ cd vattention/pod_attn/ # 安装 miniconda;如果已安装, 则跳过 $ make install_miniconda $ bash # 刷新 shell 并激活 $ conda activate pod_attn # 安装 CUDA 工具包 (pod_attn)$ conda install -y -c \ conda-forge cuda-toolkit=12.4.0 # 安装依赖项 (pod_attn)$ pip install -r requests.txt (pod_attn)$ pip install flashinfer==0.1.5 \ -i https://flashinfer.ai/whl/cu124/torch2.4 # 安装 POD-Attention 和 vAttention (pod_attn)$ make install_all
```

A.5 实验流程

POD-Attention 内核的源代码位于 vattention/pod_attn/ 文件夹中。我们的评估主要包含两种实验：注意力表现（图1、6、10、11、13、14）和端到端LLM表现（图12和表6）。图 7 使用微基准评估了各种内核融合策略。其中大多数仅需要一个 GPU，表 6 和图 12（针对 Llama-2-7B 和 Llama-3-8B）需要两个 GPU 除外。使用 vattention/pod_attn/ 文件夹中的 Makefile 运行实验，如下所示：

```
使图 1 # 2 分钟; 脚本使用的 sudo makefigure6#2 分钟 makefigure7#2 分钟 makefigure10#1 分钟; 脚本使用的 sudo makefigure11 # 2 小时 makefigure12 # 9 小时 makefigure13 # 1 分钟 makefigure14 # 1 分钟 make table6 # 4 小时
```

A.6 评价和预期结果

工件脚本将原始输出数字和日志重定向到output/文件夹，而绘制的图表可以在graphs/文件夹中找到。表以 CSV 形式保存在同一文件夹中。结果可能与论文中报告的结果存在微小的运行时间差异，但总体趋势应该保持不变。

参考

- [1] 2022. FlashAttention. <https://github.com/Dao-AI-Lab/flash-attention>
- [2] 2023. TensorRT-LLM: 用于优化大型语言模型推理的 TensorRT 工具箱. <https://github.com/NVIDIA/TensorRT-LLM>.
- [3] 2024 年。未来五年人工智能基础设施支出预计将超过 1 万亿美元。 <https://www.delloro.com/news/ai-infrastructure-spending-forecast-to-be-over-a-trillion-dollars-over-the-next-five-years/>.
- [4] 2024.ccdv/arxiv-摘要. <https://huggingface.co/datasets/ccdv/arxiv-summarization>.
- [5] 2024. CUDA C 编程指南 – 硬件实现. <https://docs.nvidia.com/cuda/cuda-c-programming-guide/#hardware-implementation>.
- [6] 2024. Llama-2-7B. <https://huggingface.co/meta-llama/Llama-2-7b-hf>.
- [7] 2024 年。合并 PR 1865: 与采样相关的关键错误修复. <https://github.com/microsoft/sarathi-serve/commit/50e59c51b85b1157e001bb8ee7a1b049d551955d#diff-450b0de5cce8a2341140afed859dc5dd3b913fa6e62d27988fcfeeacc7b33ec>
- [8] 2024. Meta-Llama-3-8B. <https://huggingface.co/meta-llama/Meta-Llama-3-8B>.
- [9] 2024 年。NVIDIA 多实例 GPU. <https://www.nvidia.com/en-us/technology/multi-instance-gpu/>.
- [10] 2024. NVIDIA 多进程服务. <https://docs.nvidia.com/deploy/mps/index.html>.
- [11] 2024.NVIDIA/cutlass: 线性代数子例程的 CUDA 模板. <https://github.com/NVIDIA/cutlass>.
- [12] 2024. 并行线程执行 ISA 版本 8.5 – 协作线程数组. <https://docs.nvidia.com/cuda/parallel-thread-execution/#cooperative-thread-arrays>.
- [13] 2024. 并行线程执行 ISA 版本 8.5 – 特殊寄存器: %smid. <https://docs.nvidia.com/cuda/parallel-thread-execution/index.html#special-registers-smid>.
- [14] 2024. 性能和调优. <https://docs.vllm.ai/en/v0.6.0/models/performance.html>.
- [15] 2024. 沙拉蒂服务. <https://github.com/microsoft/sarathi-serve>.
- [16] 2024 年。2024 年大规模人工智能基础设施状况. <https://ai-infrastructure.org/wp-content/uploads/2024/03/The-State-of-AI-Infrastructure-at-Scale-2024.pdf>.
- [17] 2024. 统一 Flash Attention 后端使用的内核. <https://github.com/vllm-project/vllm/pull/6052>.
- [18] 2024. 上游分块预填充. <https://github.com/vllm-project/vllm/issues/3130>.
- [19] 2024. vLLM: 为每个人提供简单、快速、廉价的 LLM 服务. <https://github.com/vllm-project/vllm>.
- [20] 2024. Yi-6B-200K. <https://huggingface.co/01-ai/Yi-6B-200K>.
- [21] Amey Agrawal, Junda Chen, Íñigo Goiri, Ramachandran Ramjee, Chaojie 张, Alexey Tumanov 和 Esha Choukse. 2024. Mnemosyne: 无需近似即可高效服务数百万上下文长度 LLM 推理请求的并行化策略. arXiv:2409.17264 [cs.LG] <https://arxiv.org/abs/2409.17264> [22] Amey Agrawal, Nitin Kedia, Jayashree Mohan, Ashish Panwar, Nipun Kwatra, Bhargav S Gulavani, Ramachandran Ramjee 和 Alexey Tumanov. 2024. Vidur: LLM 推理的大规模模拟框架。第七届机器学习和系统年会论文集, 2024 年, 圣克拉拉 (2024 年)。
- [23] Amey Agrawal, Nitin Kedia, Ashish Panwar, Jayashree Mohan, Nipun Kwatra, Bhargav Gulavani, Alexey Tumanov 和 Ramachandran Ramjee. 2024. 使用 Sarathi-Serve 控制 LLM 推理中的吞吐量与延迟权衡。第 18 届 USENIX 操作系统设计与实现研讨会 (OSDI 24)。USENIX 协会, 加利福尼亚州圣克拉拉, 117–134. <https://www.usenix.org/conference/osdi24/presentation/agrawal>
- [24] Amey Agrawal, Ashish Panwar, Jayashree Mohan, Nipun Kwatra, Bhargav S. Gulavani 和 Ramachandran Ramjee. 2023. SARATHI: 通过分块预填充的搭载解码进行高效的 LLM 推理. arXiv:2308.16369 [cs.LG] <https://arxiv.org/abs/2308.16369> [25] Joshua Ainslie, James Lee-Thorp, Michiel de Jong, Yury Zemlyanskiy, Federico Lebron 和 Sumit Singh. 2023. GQA: 从多头检查点训练通用多查询变压器模型。2023 年自然语言处理经验方法会议论文集, Houda Bouamor, Juan Pino 和 Kalika Bali (主编)。计算语言学协会, 新加坡, 4895–4901. <https://doi.org/10.18653/v1/2023.emnlp-main.298> [26] Jeremy Appleyard 和 Scott Yokim. 2017 年。在 CUDA 9 中对张量核心进行编程. <https://developer.nvidia.com/blog/programming-tensor-cores-cuda-9/>.
- [27] 曹诗一、刘书、泰勒·格里格斯、彼得·沙夫哈特、刘晓轩、盛英、约瑟夫·冈萨雷斯、马泰·扎哈里亚和离子·斯托伊卡。2024. MoE-Lightning: 内存受限 GPU 上的高吞吐量 MoE 推理. arXiv:2411.11217 [cs.DC] <https://arxiv.org/abs/2411.11217> [28] Li-Wen Chang, Wenlei Bao, Qi Hou, Chengquan Jiang, Ningxin Cheng, Yinminzhong, Xuanrun Zhang, Zuquan Song, Ziheng Jiang, Haibin Lin, Xin Jin 和 Xin Liu. 2024. FLUX: 通过内核融合在 GPU 上实现基于软件的快速通信重叠. arXiv:2406.06858 [cs.LG] <https://arxiv.org/abs/2406.06858> [29] Tri Dao. 2024. FlashAttention-2: 更快的注意力, 更好的并行性和工作分区。在第十二届学习表征国际会议上. <https://openreview.net/forum?id=mZn2Xyh9Ec> [30] Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra 和 Christopher Ré. 2022. FLASHATTENTION: 具有 IO 意识的快速、内存高效的精确注意力。第 36 届神经信息处理系统国际会议 (美国路易斯安那州新奥尔良) (NIPS '22) 论文集. Curran Associates Inc., 美国纽约州雷德胡克, 第 1189 条, 16 页。 [31] Tri Dao, 丹尼尔·哈齐扎、弗朗西斯科·马萨和格里戈里·西佐夫。2023. 用于长上下文推理的 Flash 解码. <https://crfm.stanford.edu/2023/10/12/flashdecoding.html>.
- [32] Kshitij Gupta, 杰夫·A 斯图尔特和约翰·D 欧文斯。2012. 针对 GPGPU 工作负载的持久线程风格 GPU 编程的研究。2012 年创新并行计算 (InPar). 1–14. <https://doi.org/10.1109/InPar.2012.6339596> [33] Connor Holmes, Masahiro Tanaka, Michael Wyatt, Ammar Ahmad Awan, Jeff Rasley, Samyam Rajbhandari, Reza Yazdani Aminabadi, Heyangqin, Arash Bakhtiari, Lev Kurilenko 和 Yuxiong He. 2024. DeepSpeed-FastGen: 通过 MII 和 DeepSpeed-Inference 为大语言模型高通量文本. arXiv:2401.08671 [cs.PF] <https://arxiv.org/abs/2401.08671> [34] 洪科, 戴国豪, 徐家明, 毛秋丽, 李秀红, 刘军, 陈康迪, 董雨涵, 王雨。2024. FlashDecoding++: 通过异步、扁平 GEMM 优化和启发式进行更快的大型语言模型推理。在《机器学习和系统学报》中, P. Gibbons, G. Pekhimenko 和 C. De Sa (编辑), 卷. 6. 148–161. https://proceedings.mlsys.org/paper_files/paper/2024/file/5321b1dabed2be188d796c21b733e8c7-Paper-Conference.pdf [35] Cunchen Hu, Heyang Huang, Liangliang Xu, Xusheng Chen, Jiang Xu, Shuang Chen, Hao Feng, Chenxi Wang, Sa Wang, Yungang Bao, Ninghui Sun 和益州山。2024. 无干扰推理: 混合下游工作负载的分解 LLM 推理. arXiv:2401.11181 [cs.DC] <https://arxiv.org/abs/2401.11181> [36] Haiyang Huang, Newsha Ardalani, Anna Sun, Liu Ke, Shruti Bhosale, Hsien-Hsin S. Lee, Carole-Jean Wu 和 Benjamin Lee. 2024. 实现专家混合的高效推理。在第三十八届神经信息处理系统年会上. <https://openreview.net/forum?id=stXtBqyTWX> [37] Abhinav Jangda, Jun Huang, Guodong Liu, Amir Hossein Nodehi Sabet, Saeed Maleki, Youshan Miao, Madanlal Musuvathi, Todd Mytkowicz,

和奥利·萨里基维。2022.打破分布式机器学习工作负载中的计算和通信抽象障碍。第27届ACM编程语言和操作系统架构支持国际会议(瑞士洛桑)(ASPLOS'22)论文集。计算机协会,美国纽约州纽约市,402–416. <https://doi.org/10.1145/3503222.3507778> [38] Abhinav Jangda, Saeed Maleki, Maryam Mehri Dehnavi, Madan Musuvathi 和 Olli Saarikivi。2024.依赖GPU内核细粒度同步的框架。2024年IEEE/ACM国际代码生成和优化研讨会论文集(英国爱丁堡)(CGO'24)。IEEE出版社,93–105. <https://doi.org/10.1109/CGO57630.2024.10444873> [39] 郝康, Srikant Bharadwaj, James Hensman, Tushar Krishna, Victor Ruhle 和 Saravan Rajmohan。2024.TurboAttention: 高吞吐量大语言模型注意力近似。arXiv:2412.08585 [cs.LG] <https://arxiv.org/abs/2412.08585> [40] Scott J. Krieder, Justin M. Wozniak, Timothy Armstrong, Michael Wilde, Daniel S. Katz, Benjamin Grimmer, Ian T. Foster 和 Ioan Raicu。2014年。用于GPU支持的多任务计算的gemtc框架的设计和评估。第23届高性能并行和分布式计算国际研讨会(加拿大不列颠哥伦比亚省温哥华)(HPDC'14)论文集。计算机协会,美国纽约州纽约市,153–164. <https://doi.org/10.1145/2600212.2600228> [41] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Shen, Lianmin Cheng, Codyhao Yu, Joseph Gonzalez, Hao Zhu, 和 Ion Stoica。2023.使用PagedAttention进行大型语言模型的高效内存管理。第29届操作系统原理研讨会论文集(德国科布伦茨)(SOSP'23)。计算机协会,美国纽约州纽约市,611–626. <https://doi.org/10.1145/3600006.3613165> [42] 敖力, 郑博坚, Gennady Pekhimenko, 范龙。2022.GPU内核的自动水平融合。2022年IEEE/ACM国际代码生成与优化研讨会(CGO)。14–27. <https://doi.org/10.1109/CGO53902.2022.9741270> [43] 梁云, Huynh Phung Huynh, Kyle Rupnow, Rick Siow Mong Goh 和 Daming Chen。2015.高效GPU时空多任务处理。IEEE Transactions on Parallel and Distributed Systems 26, 3 (2015), 748–760. <https://doi.org/10.1109/TPDS.2014.2313342> [44] Muhammad Osama, Duane Merrill, Cris Cecka, Michael Garland 和 John D. Owens。2023.Stream-K: GPU上密集矩阵-矩阵乘法的以工作为中心的并行分解。第28届ACM SIGPLAN并行编程原理与实践年度研讨会(加拿大魁北克省蒙特利尔)(PPoPP'23)论文集。计算机协会,美国纽约州纽约市,429–431. <https://doi.org/10.1145/3572848.3577479> [45] Sreepathi Pai, Matthew J. Thazhuthaveetil 和 R. Govindarajan。2013.使用弹性内核提高GPGPU并发性。第十八届编程语言和操作系统架构支持国际会议(美国德克萨斯州休斯顿)(ASPLOS'13)论文集。计算机协会,美国纽约州纽约市,407–418. <https://doi.org/10.1145/2451116.2451160> [46] Pratyush Patel, Esha Choukse, Chaojie 张, Aashaka Shah, Íñigo Goiri, Saeed Maleki 和 Ricardo Bianchini。2024.Splitwise: 使用相分裂的高效生成大语言模型。2024年ACM/IEEE第51届计算机体系结构国际研讨会(ISCAS)。118–132. <https://doi.org/10.1109/ISCA59077.2024.00019> [47] Ramya Prabhu, Ajay Nayak, Jayashree Mohan, Ramachandran Ramjee 和 Ashish Panwar。2025.vAttention: 在没有PagedAttention的情况下为LLM提供动态内存管理。第30届ACM国际编程语言和操作系统架构支持会议论文集,第1卷(荷兰鹿特丹)(ASPLOS'25)。计算机协会,美国纽约州纽约市,1133–1150. <https://doi.org/10.1145/3669940.3707256>

[48] Rya Sanovar, Srikant Bharadwaj, Renee St. Amant, Victor Rühle 和 Saravan Rajmohan。2024.精益注意力: 变压器解码阶段的硬件感知可扩展注意力机制。arXiv:2405.10480 [cs.AR] <https://arxiv.org/abs/2405.10480> [49] Jay Shah, Ganesh Bikshandi, 张英, Vijay Thakkar, Pradeep Ramani 和 Tri Dao。2024.FlashAttention-3: 具有异步和低精度的快速准确的注意力。在第三十八届神经信息处理系统年会上。 <https://openreview.net/forum?id=tVConYid20> [50] Ying Shen, Shiyi Cao, Da Cheng Li, Banghua Zhu, Zhuohan Li, Danyang Zhuo, Joseph E. Gonzalez 和 Ion Stoica。2024.服务大型语言模型的公平性。第18届USENIX操作系统设计与实现研讨会(OSDI'24)。USENIX协会,加利福尼亚州圣克拉拉,965–988. <https://www.usenix.org/conference/osdi24/presentation/sheng> [51] 宋一欣, 米泽宇, 谢浩通, 陈海波。2024.PowerInfer: 使用消费级GPU提供快速大型语言模型。ACM SIGOPS第30届操作系统原理研讨会论文集(美国德克萨斯州奥斯汀)(SOSP'24)。计算机协会,美国纽约州纽约市,590–606. <https://doi.org/10.1145/3694715.3695964> [52] Jovan Stojkovic, Chaojie 张, Íñigo Goiri, Josep Torrellas 和 Esha Choukse。2024.DynamoLLM: 设计LLM推理集群以提高性能和能源效率。arXiv:2408.00741 [cs.AI] <https://arxiv.org/abs/2408.00741> [53] 穆罕默德·瓦希布和丸山直哉。2014年。适用于内存受限GPU应用程序的可扩展内核融合。在SC'14: 高性能计算、网络、存储和分析国际会议论文集。191–202. <https://doi.org/10.1109/SC.2014.21> [54] 王世博、魏金良、Amit Sabne、Andy Davis、Berkin Ilbeyi、Blake Hechtman、陈德豪、Karthik Srinivasa Murthy、Marcello Maggioni、张俏、Sameer Kumar、Tongfei Guo、Yuanzhong Xu 和 Zongwei Zhou。2022.通过大型深度学习模型中的分解实现依赖计算的重叠通信。第28届ACM国际编程语言和操作系统架构支持会议论文集,第1卷(加拿大不列颠哥伦比亚省温哥华)(ASPLOS'2023)。计算机协会,美国纽约州纽约市,93–106. <https://doi.org/10.1145/3567955.3567959> [55] 王振宁, 杨军, Rami Melhem, Bruce Childers, 张有涛, 郭敏仪。2016.同步多内核GPU: 通过细粒度共享实现多任务吞吐量处理器。2016年IEEE高性能计算机架构国际研讨会(HPCA)。358–369. <https://doi.org/10.1109/HPCA.2016.7446078> [56] 吴波, 陈国阳, 李栋, 沉西鹏, 杰弗里·维特。2015.通过以SM为中心的程序转换在GPU上启用和利用灵活的任务分配。第29届ACM国际超级计算会议记录(美国加利福尼亚州纽波特海滩)(ICS'15)。计算机协会,美国纽约州纽约市,119–130. <https://doi.org/10.1145/2751205.2751213> [57] 吴冰阳, 刘胜宇, 钟银民, 孙鹏, 刘轩哲, 金鑫。2024.LoongServe: 通过弹性序列并行高效服务长上下文大型语言模型。ACM SIGOPS第30届操作系统原理研讨会论文集(美国德克萨斯州奥斯汀)(SOSP'24)。计算机协会,美国纽约州纽约市,640–654. <https://doi.org/10.1145/3694715.3695948> [58] 吴冰洋, 钟银民, 张自力, 黄刚, 刘轩哲, 金鑫。2023.大型语言模型的快速分布式推理服务。arXiv:2305.05920 [cs.LG] <https://arxiv.org/abs/2305.05920> [59] Hai Cheng Wu, Gregory Diamos, Srihari Cadambi 和 Sudhakar Yalamanchili。2012. Kernel Weaver: 自动融合数据库基元以实现高效GPU计算。2012年第45届IEEE/ACM国际微架构研讨会。107–118. <https://doi.org/10.1109/MICRO.2012.19>

- [60] 叶子豪、陈乐群、赖瑞航、林武伟、张伊能、斯蒂芬妮·王、陈天琪、Baris Kasikci、Vinod Grover、Arvind Krishnamurthy、Luis Ceze。2025. FlashInfer: 用于 LLM 推理服务的高效且可定制的注意力引擎。arXiv:2501.01005 [cs.DC] <https://arxiv.org/abs/2501.01005> [61] Tsung Tai Yeh, Amit Sabne, Putt Sakdhnagool, Rudolf Eigenmann 和 Timothy G. Rogers。2017. Pagoda: 针对窄任务的细粒度 GPU 资源虚拟化。第 22 届 ACM SIGPLAN 并行编程原理与实践研讨会论文集 (美国德克萨斯州奥斯汀) (PPoPP '17)。计算机协会, 美国纽约州纽约市, 221–234。 <https://doi.org/10.1145/3018743.3018754> [62] Gyeong-In Yu, Joo Seong Jeong, Geon-Woo Kim, Soojeong Kim 和 Byung-Gon Chun。2022. Orca: 基于 Transformer 的生成模型的分布式服务系统。第 16 届 USENIX 操作系统设计与实现研讨会 (OSDI 22)。USENIX 协会, 卡尔斯巴德, 加利福尼亚州, 521–538。 <https://www.usenix.org/conference/osdi22/presentation/you> [63] 赵涵, 崔伟豪, 陈全, 郭敏仪。2023. ISPA: 通过细粒度资源管理利用 GPU 中的 SM 内并行性。IEEE 传输。计算。72, 5 (2023), 1473–1487。 <https://doi.org/10.1109/TC.2022.3214088> [64] 郑连民, 尹良胜, 谢志强, 孙初跃, 黄杰夫, 于浩, 曹诗怡, Christos Kozyrakis, Ion Stoica, Joseph E. Gonzalez, Clark Barrett, 盛英。2024. SGLang: 结构化语言模型程序的高效执行。在第三十八届神经信息处理系统年会上。 <https://openreview.net/forum?id=VqkAKQibpq> [65] Yinminzhong, Shengyu Liu, Junda Chen, Jianbo Hu, Yibo Zhu, Xu-anzhe Liu, Xin Jin, and Hao Zhang。2024. DistServe: 分解预填充和解码以实现 Goodput 优化的大型语言模型服务。第 18 届 USENIX 操作系统设计与实现研讨会 (OSDI 24)。USENIX 协会, 加利福尼亚州圣克拉拉, 193–210。 <https://www.usenix.org/conference/osdi24/presentation/zhong-yinmin> [66] Kan Zhu, Yilong Zhao, Yangyu Zhu, Gefei Zuo, Yile Gu, Dedong Xie, Yufei Gau, Qingyu Xu, Tian Tang, Zihao Ye, Keisuke Kamahori, Chien-Yu Lin, Stephanie Wang, Arvind 克里希那穆蒂和巴里斯·卡西奇奇。2024. NanoFlow: 迈向服务吞吐量的最佳大型语言模型。arXiv:2408.12757 [cs.DC] <https://arxiv.org/abs/2408.12757>